

Laravel Internship Weekly Task

Week Topic: Eloquent ORM, Relationships & Resource Controllers

In the previous task, interns connected Laravel to a database and worked with basic queries. This week, interns will move beyond simple queries and learn the proper Laravel way of handling data using **Eloquent ORM**. They will design model relationships, build a resourceful controller for full CRUD operations, validate form input, and create a clean Blade interface with layouts and components. The objective is to teach interns to write clean, maintainable, Laravel-standard code instead of relying on raw SQL.

Topics Covered

- Eloquent Models & Mass Assignment
- Eloquent CRUD methods (create, find, update, delete)
- One-to-Many and Many-to-Many Relationships
- Eager Loading with **with()**
- Seeders & Factories using Faker
- Resource Controllers & Route Model Binding
- Form Validation & Form Requests
- Blade Layouts, Components & Partials
- Pagination & Search using Eloquent
- Flash messages & Sessions

Main Project Task

Interns must **extend** the existing Student Result Management System from the previous task. Replace any raw queries or DB facade calls with proper **Eloquent Models and Relationships**. The system must support full CRUD operations for students, subjects, and marks — with validation, search, pagination, and a properly structured Blade UI using layouts.

Project Requirements

Component	Requirement
Eloquent Models	Student, Subject, Mark with relationships defined
Seeders & Factories	Seed 10+ students and 5+ subjects using Faker
Resource Controller	StudentController with all 7 resourceful methods
Route Model Binding	Use type-hinted model parameters in routes
Form Validation	Validate every input with proper error messages
Blade Layout	Master layout with @extends, @section, @yield

Blade Components	Reusable components for cards, alerts, forms
Pagination	Paginate students list — 10 per page
Search	Search students by name or roll number

Task 1 — Eloquent Models & Mass Assignment

Interns must learn to:

- Use **php artisan make:model**
- Define **\$fillable** and **\$guarded** properties
- Use Eloquent methods: **create()**, **find()**, **where()**, **update()**, **delete()**
- Use **php artisan tinker** to test models

Requirements:

- 1 Create Student, Subject, and Mark models
- 2 Define **\$fillable** arrays for each model
- 3 Replace all raw DB queries from previous task with Eloquent methods
- 4 Test each model in tinker — create, fetch, update, delete one record
- 5 Use **firstOrCreate()** and **updateOrCreate()** at least once

Task 2 — Eloquent Relationships

Interns must implement:

- **hasMany()** and **belongsTo()** — one-to-many
- **belongsToMany()** — many-to-many with pivot table
- Accessing related records: **\$student->marks**
- Eager loading using **with()** to prevent N+1 queries

Requirements:

- 1 Student **hasMany** Marks
- 2 Mark **belongsTo** Student and **belongsTo** Subject
- 3 Student **belongsToMany** Subjects through marks pivot
- 4 Fetch all students with their marks using **Student::with('marks')->get()**
- 5 Display each student's subject-wise marks in the Blade view

Task 3 — Seeders & Factories

Interns must create:

- **StudentFactory** using the Faker library
- Seeders for students, subjects, and marks
- A **DatabaseSeeder** that runs all seeders together

Exercise Questions:

- 1 Generate 20 students with realistic names and emails using Faker
- 2 Seed 5 subjects (Math, English, Physics, Chemistry, Computer)
- 3 For every student, generate marks in each subject (random 30–100)
- 4 Run **php artisan migrate:fresh --seed** and verify data

Task 4 — Resource Controller & Route Model Binding

Interns must implement:

- **php artisan make:controller StudentController --resource**
- **Route::resource('students', StudentController::class)**
- All 7 resourceful methods: index, create, store, show, edit, update, destroy
- Type-hinted Route Model Binding instead of **Student::find(\$id)**

Requirements:

- 1 List all students on **/students**
- 2 Show single student details on **/students/{student}**
- 3 Add new student via form on **/students/create**
- 4 Edit existing student on **/students/{student}/edit**
- 5 Delete student with confirmation
- 6 Use Route Model Binding — no manual find() calls
- 7 Show success flash messages after every action

Task 5 — Form Validation

Interns must practice:

- **\$request->validate([...])**
- Rules: required, email, unique, numeric, min, max, between
- Displaying errors with **@error** directive
- Preserving old input with **old('field')**
- Creating a custom **Form Request** class

Requirements:

- 1 Validate name (required, min 3 chars, max 50)
- 2 Validate email (required, valid email, unique:students)
- 3 Validate roll number (required, unique, numeric)
- 4 Validate attendance (numeric, between 0–100)
- 5 Show all error messages in red below each input
- 6 Re-fill form fields after validation failure
- 7 Create a **StudentRequest** Form Request class for the rules

Task 6 — Blade Layouts & Components

Interns must practice:

- **@extends**, **@section**, **@yield** for master layouts
- **@include** for partial files (navbar, footer)
- Blade Components via **php artisan make:component**
- Passing data to components with attributes and slots

Requirements:

- 1 Create **layouts/app.blade.php** with navbar and footer
- 2 All pages must extend the master layout
- 3 Create a **<x-alert>** component for flash messages
- 4 Create a **<x-student-card>** component for displaying a student
- 5 Create a reusable form partial for create & edit pages

Task 7 — Search & Pagination

Interns must implement:

- Search students by name or roll number using **where()** and **like**
- Filter by grade (A, B, C, F) using a query parameter
- Sort by name, marks, or attendance
- **paginate(10)** on the listing page
- Display pagination links in Blade with **{{ \$students->links() }}**

Final Dashboard Requirement

Interns must combine all concepts into a complete **Eloquent-Powered Student Result Dashboard**.

Dashboard Features:

- Student listing with pagination & search
- Add / Edit / Delete student with validation
- Student detail view showing all subjects & marks
- Filter by grade (A, B, C, F)
- Summary statistics: total students, average percentage, pass/fail count
- Topper of the class card on the dashboard
- Flash messages on every successful action
- Clean Blade layout with reusable components

Submission Guidelines

Requirement	Mandatory
-------------	-----------

GitHub Repository	Yes
Updated Laravel Project	Yes
All raw queries replaced with Eloquent	Yes
Migrations & Seeders working	Yes
Blade layout & components	Yes
Form validation on all forms	Yes
Proper Code Comments	Yes
README with setup & screenshots	Yes

Important Instructions

- Use Eloquent for **every** database operation — no raw queries allowed
- Use **php artisan** commands — do not create files manually
- Follow Laravel naming conventions (singular model, plural table)
- Keep business logic inside controllers — never in Blade
- Always use Route Model Binding instead of **find()**
- Add comments above every important method
- Push code to GitHub daily — at least one meaningful commit per task
- Include a README.md with setup steps and feature screenshots

Interns are required to complete this assignment using their own logic and understanding. Direct use of AI-generated solutions is strictly prohibited. Understanding of every line of code will be verified during the weekly review.